

COVID-19 Cough Detection Web Application

Software Design Document

Version 1.0

Drew Quashie, Amanda Ogbonna, Loleyi Oluwatomisin, Richard Alonso Garcia Faculty
Advisor: Dr. Zahra Nematzadeh

Table of Contents

1.0 Introduction
2.0 System Overview
3.0 System Architecture
4.0 Data Design
5.0 Component Design
6.0 Human Interface Design
7.0 Algorithm Design
8.0 Performance and Optimization
9.0 References

1.0 Introduction

1.1 Purpose

This software design document describes the architecture and system design of the COVID-19 Cough Detection Web Application, a browser-based solution for preliminary COVID-19 risk pre-screening from three guided cough recordings, gated by a cough-presence classifier, analyzed with server-side convolutional neural network inference, and tracked on-device over time.

1.2 Background and Objectives

During pandemic surges, individuals with mild or uncertain symptoms often visit testing centers for reassurance, overloading laboratories and clinical staff. Early, accessible pre-screening can direct people who are likely negative toward self-monitoring while surfacing likely positive cases to care sooner.

The primary objectives are:

- Provide a pre-screening risk assessment from three guided cough recordings using ordinary device microphones, no specialized hardware.
- Deliver near-real-time feedback from browser capture to displayed result.
- Support longitudinal self-tracking so users (and, via sharing, clinicians) can observe trends over time.
- Commit to honest result framing: a pre-screening signal, never a diagnosis.

1.3 Definitions and Acronyms

Acronym	Term
CNN	Convolutional Neural Network
MFCC	Mel-Frequency Cepstral Coefficients
STFT	Short-Time Fourier Transform
SNR	Signal-to-Noise Ratio
API	Application Programming Interface
ONNX	Open Neural Network Exchange
UI/UX	User Interface / User Experience

Term	Definition
Risk Level	Categorized pre-screening assessment (Low / Medium / High) derived from model output
Confidence Score	Mean sigmoid signal strength of the winning class across the 3 coughs, indicating prediction certainty

Term	Definition
Segment	Fixed-length (1 s) excerpt extracted from one cough event in the user's recording
Cough Event	One detected cough within the recording (three expected per session)
Vote	Per-cough class decision aggregated into the final class (see §7.3)
Feature Map	Matrix of MFCC or Mel-spectrogram coefficients derived from one segment
Inconclusive	Result withheld because input quality or model confidence is below threshold

2.0 System Overview

The application comprises four major subsystems that collaborate to deliver the complete experience.

2.1 Audio Capture Module

Handles all aspects of microphone acquisition in the browser via the Web Audio API and `getUserMedia`: permission request, live input-level metering, recording start/stop, and capture guidance. One capture session covers **three coughs**: the app cues the user to cough three times, uses onset detection to confirm each cough as it happens (○ ○ ○ progress filling per detected cough), and uploads a single recording. Basic client-side checks (silence detection, gross clipping) provide immediate feedback; authoritative validation happens server-side, starting with the cough-presence gate.

2.2 Machine Learning Inference Engine

The core analytical component. The FastAPI backend receives the raw upload and runs, in order:

1. **Cough-presence gate**, a pretrained XGBoost cough/no-cough classifier (from the CoughVid GitHub repository) decides whether the clip contains cough audio at all; failure returns an Inconclusive result *before* any segmentation.
2. **Segmenter**, locates the three cough events and cuts one fixed 1 s window per cough.
3. **CNN inference (×3)**, each segment's joint MFCC + Mel-spectrogram features pass through a PyTorch CNN/ResNet classifier whose head is **3 logits with sigmoid activation** (Negative / Uncertain / Positive); per-cough class = argmax of the three sigmoid outputs.
4. **Deterministic aggregation**, majority vote across the three per-cough classes; a 1-1-1 split falls back to argmax of the mean sigmoid vector (see §7.3).

2.3 Results Processing and Display

Interprets raw model output into user-facing information: aggregates the three per-cough votes into a final class, maps that class to a Low/Medium/High risk level with a minimum-confidence gate, generates appropriate clinical recommendations, and formats results with mandatory disclaimers. Renders the longitudinal dashboard from locally stored history.

2.4 Data Management System

Responsible for browser-side persistence of analysis history, trend series, and user preferences using IndexedDB. All health data remains on the user's device; nothing is retained on the server after inference completes. The system supports manual export (CSV/JSON) so a user can share their trend record with a clinician.

3.0 System Architecture

3.1 Architectural Design

The system follows a three-tier client–server architecture with a browser device layer:

PRESENTATION LAYER (React, browser) Dashboard · Recorder · Results · Trends · Settings	
BROWSER DEVICE LAYER Web Audio API · getUserMedia · IndexedDB	
APPLICATION LOGIC (FastAPI)	REST API (HTTPS, multipart) POST /analyze
ML LAYER Cough gate (XGBoost) · Segmenter (×3) · Feature extract (×3) · CNN classifier (×3) · Vote aggregation	DATA PERSISTENCE (server) Stateless, no health data stored post-inference

The Presentation Layer owns all user interaction and local persistence. The Application Logic Layer validates requests and coordinates the ML pipeline. The ML Layer performs analytical computation in the order gate → segment → classify → vote. The server is **stateless with respect to health data**: an inference request in, a result out, nothing retained.

3.2 Decomposition Description

- React ↔ FastAPI:** single primary endpoint, `POST /analyze`, accepts multipart audio (WAV/OGG/WEBM, one recording of 3 coughs) and returns JSON: final risk level, confidence, `vote` summary (winner and vote counts), `per_cough` breakdown ($3 \times$ class + sigmoid vector), cough-gate result, and trace ID. Versioned under `/v1`.

- **FastAPI ↔ ML Layer:** model loaded once at process start; the API layer never touches tensors directly beyond passing the preprocessed payload to the inference service.
- **React ↔ IndexedDB:** all history and trend reads/writes go through the Storage Manager; no other component touches IndexedDB.
- **Future boundary:** the ML Layer's feature-extraction + classifier pair is intentionally kept free of FastAPI dependencies so it can be exported to ONNX and embedded in the browser (see §8.2).

4.0 Data Design

4.1 Data Description

The system manages four primary data types:

- **Audio Data:** raw microphone capture of one session containing 3 coughs, in browser-native formats (WEBM/OGG, transcoded or accepted as WAV), typically 5–15 s and under 1 MB. Used transiently: uploaded, analyzed, discarded by the server; optionally kept locally at user discretion.
- **Feature Tensors:** per-cough MFCC and Mel-spectrogram matrices computed from each of the three 1 s segments, transient in memory on the server only, never persisted.
- **Model Data:** trained PyTorch weights (`.pt`) with a planned ONNX export (`.onnx`) for the future client-side path, plus the pretrained XGBoost cough detector artifact from the CoughVid repository.
- **Analysis Records:** structured JSON records, timestamp, risk level, vote breakdown (3 per-cough classes + sigmoid vectors), confidence, cough-gate/quality metrics, and optional user notes, stored in IndexedDB on the user's device.

4.2 Data Dictionary

Data Element	Format	Size	Description
Raw Cough Audio	WEBM/WAV	100 KB–1 MB	One recording containing 3 guided coughs (~5–15 s)
Quality Report	JSON	<2 KB	Cough-gate score, SNR, clipping ratio, per-cough onset times
Feature Tensors (MFCC + Mel)	float32 arrays	~100–300 KB each	3 × 1 s segment feature maps, server RAM only

Data Element	Format	Size	Description
Trained Model	.pt	10–90 MB	Full PyTorch checkpoint (training only)
Inference Model	.onnx	5–50 MB	Exported CNN weights (serving / future browser use)
Cough Detector	pretrained XGBoost (CoughVid repo)	KB–MB	Cough/no-cough gate before segmentation
Analysis Record	JSON (IndexedDB)	<2 KB	Result metadata + vote breakdown + quality metrics + user note
User Preferences	JSON (IndexedDB)	<2 KB	Retention settings, guidance dismissal flags

4.3 Data Storage Design

IndexedDB object stores:

Store	Key	Value
analyses	auto-increment id	Analysis Record (indexed by timestamp)
preferences	fixed key	User Preferences

Retention is user-configurable (default: keep records indefinitely until manually deleted; bulk-delete and export actions provided). The server keeps **no** copy of audio, features, or results.

5.0 Component Design

5.1 Presentation Layer Components

Main UI Component - Responsibility: application navigation and dashboard rendering. - Key methods: - `navigateToScreen(screenName)`, handles view transitions. - `displayDashboard()`, renders recent analyses summary and entry points. - `handleUserPreferences()`, settings for retention, export, guidance.

Recorder UI Component - Responsibility: microphone interface guiding a 3-cough capture session. - Key methods: - `initializeRecorder()`, requests `getUserMedia`, wires Web Audio context. - `showLiveLevelMeter()`, visual input meter confirming mic activity. - `startRecording()`, begins one capture session; displays cue prompt "cough 3 times". - `detectCoughCue()`, client-side onset detection fills the `○ ○ ○` progress as each cough is heard; stops automatically after the third or on user stop. - `uploadForAnalysis(blob)`, single POST of the full recording to `/v1/analyze`, manages loading and error states.

Results UI Component - Responsibility: presentation of assessment outcome. - Key methods: - `displayRiskAssessment(result)`, Low/Medium/High badge, confidence bar, vote summary and per-cough breakdown. - `presentMedicalDisclaimer()`, mandatory pre-screening-only notice. - `generateRecommendations(riskLevel)`, actionable guidance per level. - `saveLocally(result)`, writes Analysis Record (including vote breakdown) via Storage Manager.

Trends UI Component - Responsibility: longitudinal dashboard. - Key methods: - `renderTrendChart(records)`, risk level and confidence over time. - `exportHistory(format)`, CSV/JSON export for clinician sharing.

5.2 Application Logic Layer Components

Analysis Request Controller - Responsibility: HTTP contract and validation. - Key methods: - `acceptUpload(request)`, size/type/duration limits, abuse controls. - `validateRequest(audioMeta)`, reject empty, oversized, or truncated payloads before ML work begins. - `formatResponse(result)`, assemble JSON contract, map internal errors to client-safe messages.

Inference Orchestrator - Responsibility: coordinates the ML pipeline end to end. - Key methods: - `executeAnalysisPipeline(audio)`, cough gate → locate/cut 3 segments → feature extraction ×3 → CNN inference ×3 → vote aggregation → risk mapping. - `monitorPerformanceMetrics()`, per-stage latency logging. - `handlePipelineErrors()`, gate failure returns `Inconclusive("cough not detected")` as a 200-level result with reason; system problems return 5xx.

Result Processor - Responsibility: interpretation and formatting of model output. - Key methods: - `interpretModelOutput(voteResult)`, aggregated vote + mean sigmoid vector → risk level. - `applyConfidenceGate(confidence)`, below-threshold → `Inconclusive`. - `formatForDisplay(result)`, assemble the user-facing result object including vote and `per_cough`.

5.3 Machine Learning Layer Components

Model Loader - Responsibility: model lifecycle on the server. - Key methods: - `loadModel()`, load .pt (or .onnx) once at startup. - `verifyModelIntegrity()`, checksum + smoke-test inference on a reference tensor. - `optimizeInferenceSettings()`, `eval()` mode, `torch.inference_mode()`, optional half-precision on GPU.

Cough Presence Detector - Responsibility: reject non-cough audio *before* any segmentation work. - Key methods: - `loadDetector()`, load the pretrained XGBoost cough/no-cough classifier shipped with the CoughVid repository (no training on our side). - `detectCough(clip)`, returns pass/fail + score; fail short-circuits the pipeline to `Inconclusive("cough not detected")`.

Audio Preprocessor - Responsibility: raw clip → three analyzable segments. - Key methods: - `validateQuality(clip)`, SNR estimate, clipping ratio, duration bounds; fails → Inconclusive with reason. - `detectCoughEvents(clip)`, energy-based onset detection locating the cough events within the recording (expected: 3). - `extractSegments(clip)`, cut one fixed 1 s window per cough event, onset-centered (window length from the 98.5th-percentile length experiment; see §7.1); pad events shorter than 1 s. - `normalize(segment)`, amplitude normalization and resampling to the training sample rate.

Feature Extractor - Responsibility: segment → model input (runs once per segment, ×3). - Key methods: - `computeMelSpectrogram(segment)`, log-Mel power spectrogram. - `computeMFCCs(segment)`, cepstral coefficients from log-Mel energies. - `fuseFeatures(mel, mfcc)`, stack into the joint representation the classifier consumes (fusion point fixed at training time; see §7.2).

Prediction Interpreter - Responsibility: model output → assessment. - Key methods: - `applySigmoid(logits)`, elementwise sigmoid over the 3 output logits (independent sigmoids; they do **not** sum to 1). - `classifyCough(logits)`, per-cough class = argmax of the 3 sigmoid outputs. - `aggregateVotes(perCough)`, majority vote across the 3 coughs; 1-1-1 split → argmax of the mean sigmoid vector. - `calculateConfidence(vote)`, mean sigmoid value of the winning class across the 3 coughs; used by the confidence gate. - `assignRiskLevel(vote, meanSigmoid)`, risk level from the vote winner (Negative→LOW, Uncertain→MEDIUM, Positive→split on mean Positive signal ≥ 0.65 →HIGH else MEDIUM); see §7.3.

5.4 Device and Persistence Layer Components

Microphone Interface (browser) - Responsibility: device audio access abstraction. - Key methods: - `acquireMicPermission()`, `getUserMedia` with graceful denial handling. - `configureCapture()`, sample rate and channel selection. - `releaseMic()`, teardown after stop or navigation.

Storage Manager (browser) - Responsibility: local persistence and retrieval. - Key methods: - `saveAnalysis(record)`, IndexedDB insert. - `retrieveHistory(range)`, records for dashboard and trends. - `exportHistory(format)`, CSV/JSON for sharing. - `pruneHistory(policy)`, retention enforcement.

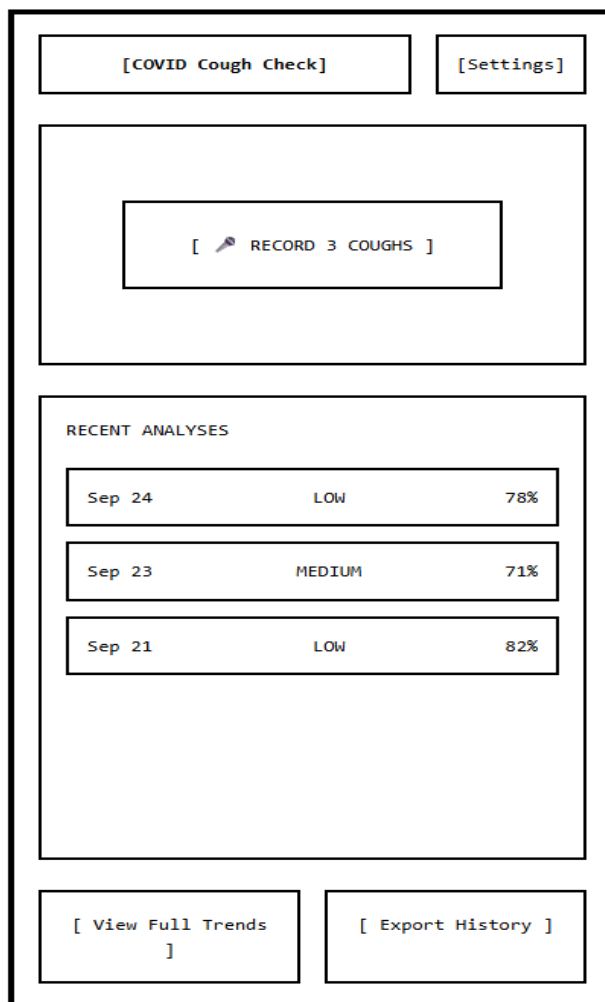
6.0 Human Interface Design

6.1 Overview of User Interface

The interface is a responsive single-page React application prioritizing a clear information hierarchy: capture is one tap away, results are legible at a glance, and trends are always available but never alarming. All result screens carry the pre-screening disclaimer; risk levels are conveyed by color **and** text (accessibility), never color alone.

6.2 Screen Images and Mockups

Main Dashboard Screen



Recorder Screen

[Back]

Record 3 Coughs

[Help]

Give 3 loud, clear coughs, one at a time.
Hold the device 6-8 inches away in a quiet room.

Cue progress: • • ○ (2 of 3)

• • • • •

(LIVE LEVEL METER)

[RECORD / STOP]

Tip: if the meter doesn't move, check microphone permissions.

Results Screen

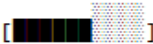
[Back]

Analysis Complete

[Save] [Share]

IMPORTANT: PRE-SCREENING TOOL ONLY – NOT A DIAGNOSIS. ALWAYS CONFIRM WITH AN OFFICIAL COVID-19 TEST.

Result: MEDIUM RISK

Confidence: 62% 

Vote: 2 of 3 coughs → POSITIVE

Cough 1: Positive | Cough 2: Positive |
Cough 3: Negative

Signal strength (mean sigmoid):
Negative 0.31|Uncertain 0.18|Positive 0.62

Recommendations:

- Consider taking an official test
- Wear a mask around others until confirmed
- Monitor symptoms over the next 48 hours

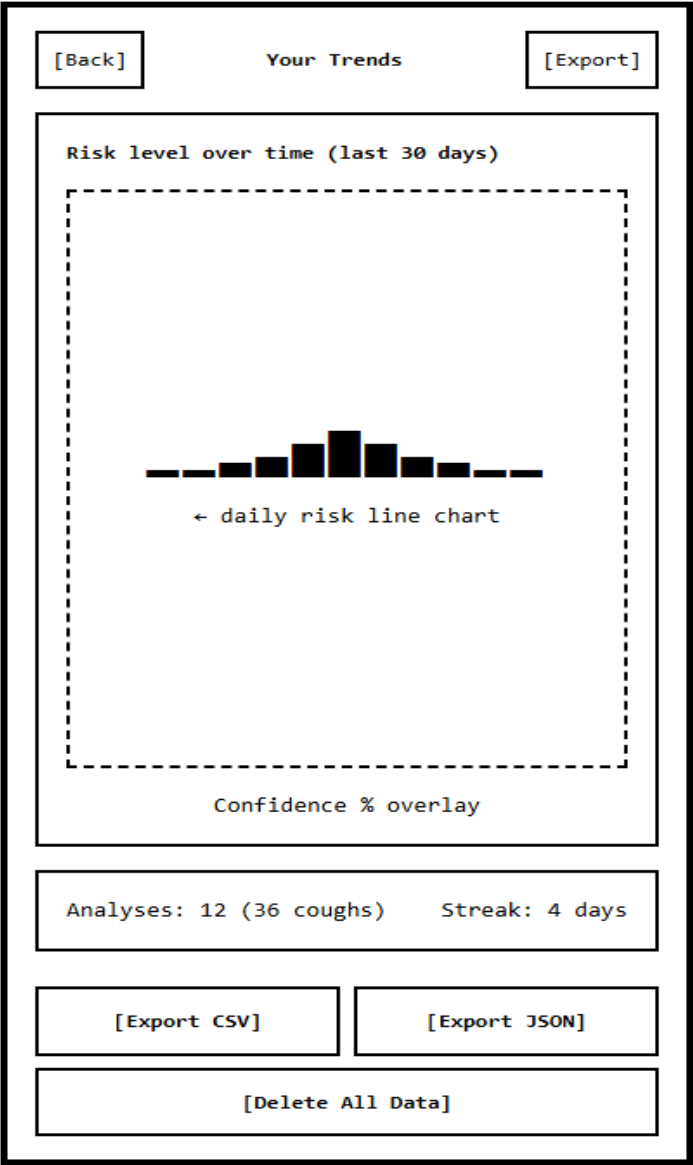
Quality: cough detector pass • SNR good

[Save Result]

[New Analysis]

[Learn More]

Longitudinal Trends Screen



7.0 Algorithm Design

7.1 Audio Processing Pipeline

BEGIN Audio Processing Algorithm

1. RECEIVE raw audio clip from client upload
(one session containing 3 coughs)
2. VALIDATE clip bounds: duration, byte size, decodable format
3. VALIDATE quality metrics:
 - Estimate SNR (spectral gating noise floor)
 - Check clipping ratio
4. IF quality inadequate THEN RETURN Inconclusive(reason)
5. COUGH GATE: run pretrained XGBoost cough/no-cough classifier (CoughVid repo artifact) on the clip
6. IF cough not detected THEN
RETURN Inconclusive("cough not detected")
7. LOCATE cough events on the timeline
 - Energy-based onset detection; expect 3 events
8. FOR EACH cough event:
 - CUT fixed 1-second onset-centered window
(segment length = 98.5th percentile of CoughVid cough durations ≈ 0.98 s, rounded to 1.0 s)
IF event shorter than 1 s THEN pad with silence
 - RESAMPLE to training sample rate (22.05 kHz mono)
 - NORMALIZE amplitude to $[-1, 1]$
 - EXTRACT log-Mel spectrogram and MFCCs
 - APPLY training-time normalization (mean/std)
9. RETURN list of 3 fused feature tensors

END Audio Processing Algorithm

7.2 Model Training and Label Engineering

Output layer. The CNN head is **3 logits with sigmoid activation** (Negative / Uncertain / Positive), one per class, applied elementwise. Sigmoids are independent, outputs do not sum to 1, so they are treated as per-class *signal strengths*, not a probability distribution. The per-cough predicted class is `argmax(sigmoid(logits))`.

The label problem (open design item). CoughVid provides COVID-positive / COVID-negative style status labels plus metadata (symptoms, age range, gender, etc.), but the model requires three classes: Negative, Uncertain, Positive. The third class does not exist natively. The intended construction:

- **Symptom-derived third class:** use CoughVid metadata to label recordings with inconclusive or mixed symptom reports (neither confirmed COVID+ nor clean negative) as *Uncertain*, and train a true 3-way head. Risk: noisy labels, since symptom metadata is self-reported, **flagged for advisor review**, with no fallback decided yet.

```

BEGIN Training Algorithm
INPUT: CoughVid clips + labels (3rd class from metadata, $above)
1. FILTER clips: duration bounds, SNR floor, dedupe
2. SPLIT train/val/test by speaker (no speaker leakage)
3. FOR each training clip:
    - segment to 1 s (per §7.1)
    - AUGMENT (random, per-epoch):
        TimeMasking, FrequencyMasking, TimeStretch,
        additive ambient noise, pitch shift
    - EXTRACT joint MFCC + Mel features
4. TRAIN CNN/ResNet: 3-logit head, sigmoid activation,
    per-logit binary cross-entropy, class-weighted loss
    (handles residual class imbalance)
5. EVALUATE on held-out test split:
    Accuracy, Precision, Recall, F1, ROC-AUC
6. CALIBRATE confidence (temperature scaling) so sigmoid
    magnitudes are meaningful for the confidence gate
7. EXPORT best checkpoint to ONNX
END Training Algorithm

```

7.3 Risk Classification Algorithm

```

BEGIN Risk Classification Algorithm
INPUT: logits[3][3]    (3 coughs x 3 class logits),
       cough_gate, quality_report

IF quality_report failed THEN
    RETURN "INCONCLUSIVE - " + quality_report.reason
END IF
IF cough_gate failed THEN
    RETURN "INCONCLUSIVE - cough not detected"
END IF

-- Per-cough classification (3x)
FOR i IN 0..2:
    sig_i      = sigmoid(logits[i])           -- elementwise
    class_i    = ARGMAX(sig_i)               -- Negative/
                                           -- Uncertain/Positive
END FOR

-- Deterministic aggregation (3x votes)
votes = COUNT(class_i)
winner = MAJORITY(votes)
IF votes split 1-1-1 THEN
    winner = ARGMAX( MEAN(sig_i) )           -- tie-break
END IF

-- Confidence gate
confidence = MEAN( sig_i[winner] )          -- across 3 coughs
IF confidence < 0.50 THEN
    RETURN "INCONCLUSIVE - low model confidence"
END IF

-- Risk mapping from the vote winner, with the mean
-- Positive signal deciding severity inside Positive
p_pos_mean = MEAN( sig_i[Positive] )

```

```

IF winner == Negative THEN
    risk = "LOW"
    recommendation = "Continue normal precautions; re-check if
                      symptoms develop"
ELSE IF winner == Uncertain THEN
    risk = "MEDIUM"
    recommendation = "Consider testing; mask up around others;
                      monitor symptoms over the next 48 hours"
ELSE -- winner == Positive
    IF p_pos_mean >= 0.65 THEN
        risk = "HIGH"
        recommendation = "Take an official COVID-19 test soon and
                          contact a healthcare provider if symptomatic"
    ELSE
        risk = "MEDIUM"
        recommendation = "Consider testing; mask up around others;
                          monitor symptoms over the next 48 hours"
    END IF
END IF

RETURN (risk, recommendation, votes, sig vectors, confidence)
END Risk Classification Algorithm

```

(Risk level is a pure function of the vote winner plus `p_pos_mean`, displayed class and displayed risk can never contradict each other. The HIGH/MEDIUM split at 0.65 is the only tunable threshold; final value set by calibration.)

8.0 Performance and Optimization

8.1 Performance Targets

- **Capture to result:** < 6 seconds end-to-end (3-cough capture is user-driven; upload + gate + 3× segmentation/inference + response) on a typical broadband connection.
- **Server inference:** < 1.5 seconds per analysis total, covering the XGBoost gate and **3 forward passes**, excluding network transfer.
- **API payload:** < 1 MB upload per analysis (~5–15 s clip), < 10 KB JSON response (includes `per_cough` breakdown).
- **Concurrent analyses:** sustain 10 concurrent inference requests without failure on target hardware.
- **Client footprint:** initial bundle < 3 MB compressed; history of 1 year (≈365 records) < 2 MB in IndexedDB.
- **Availability of local history:** dashboard renders from IndexedDB in < 500 ms regardless of network state (history works offline).

8.2 Optimization Strategies

Inference efficiency: load the model once at process start; run under `torch.inference_mode()`; export to ONNX and serve via ONNX Runtime for CPU consistency; keep preprocessing (feature extraction) in the same worker to avoid serialization overhead.

Pipeline latency: accept compressed browser audio directly (avoid client-side re-encode to WAV); run quality gating before feature extraction so bad inputs fail fast; log per-stage timings against the 1.5 s inference budget.

Future client-side inference (noted, not committed): the ML Layer is kept free of FastAPI dependencies specifically so the ONNX export can later run in-browser via ONNX Runtime Web, eliminating upload latency entirely and enabling fully offline analysis. This is a post-v2 path; the v2 architecture is server-side as documented in §3.1.

Storage: single JSON-blob records per analysis keep IndexedDB access simple; export duplicates data out rather than requiring a sync protocol. Retention pruning runs lazily on app start.

9.0 References

1. CoughVid, "Coughvid Dataset," ETH Zurich, 2020, <https://github.com/coughlanb/CoughVID>.
2. "Convolutional Neural Network," Wikipedia, Wikimedia Foundation, 2023, https://en.wikipedia.org/wiki/Convolutional_neural_network.
3. PyTorch, "PyTorch Documentation," Meta AI, 2024, <https://pytorch.org/docs/stable/>.
4. FastAPI, "FastAPI Documentation," 2024, <https://fastapi.tiangolo.com/>.
5. Su, Y., Zhang, Y., Wang, P., & Madani, K. (2019). Environment Sound Classification Using a Two-Stream CNN with Decision-Level Fusion. *Sensors*, 19(7), 1733. <https://doi.org/10.3390/s19071733>
6. Qu, Y., Li, S., Qin, Z., & Lu, X. (2022). Acoustic scene classification based on three-dimensional multi-channel feature-correlated deep learning networks. *Scientific Reports*, 12, 17863. <https://doi.org/10.1038/s41598-022-17863-z>
7. Karim, S. H. T., et al. (2024). A Multi-Feature Fusion Approach for Dialect Identification using 1D CNN. *JOIV*, 8(3), 1246. <https://doi.org/10.62527/joiv.8.3.2146>
8. Chu, H.-C., Zhang, Y.-L., & Chiang, H.-C. (2023). A CNN Sound Classification Mechanism Using Data Augmentation. *Sensors*, 23(15), 6972. <https://doi.org/10.3390/s23156972>
9. Aggarwal, A., et al. (2022). Two-Way Feature Extraction for Speech Emotion Recognition. *Sensors*, 22(6), 2378. <https://doi.org/10.3390/s22062378>
10. World Health Organization, "COVID-19 testing," 2023, <https://www.who.int/emergencies/diseases/novel-coronavirus-2019/situation-reports>.